

Implementing Graph Traversals

1 Data Structures for Graphs

- representing graphs
- adjacency matrices and adjacency lists

2 Implementing the Breadth-First and Depth-First Search Algorithms

- implementing the breadth-first search algorithm
- implementing the depth-first search algorithm

3 Applications

- connected components
- bipartiteness testing

CS 401/MCS 401 Lecture 4
Computer Algorithms I
Jan Verschelde, 24 June 2026

Implementing Graph Traversals

1 Data Structures for Graphs

- representing graphs
- adjacency matrices and adjacency lists

2 Implementing the Breadth-First and Depth-First Search Algorithms

- implementing the breadth-first search algorithm
- implementing the depth-first search algorithm

3 Applications

- connected components
- bipartiteness testing

graph representations

A graph $G = (V, E)$ has two dimensions:

- 1 n is the number of vertices, $n = \#V$,
- 2 m is the number of edges, $m = \#E$.

What is the relation between n and m ?

- If G is a tree, then $m = n - 1$.
- If G is *completely connected*, that is:
there is an edge between every pair of vertices, then

$$m = \binom{n}{2} = \frac{n(n-1)}{2} = O(n^2).$$

In a connected graph, m ranges between $O(n)$ and $O(n^2)$.

Implementing Graph Traversals

1 Data Structures for Graphs

- representing graphs
- adjacency matrices and adjacency lists

2 Implementing the Breadth-First and Depth-First Search Algorithms

- implementing the breadth-first search algorithm
- implementing the depth-first search algorithm

3 Applications

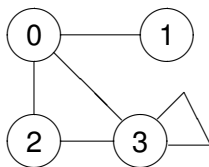
- connected components
- bipartiteness testing

adjacency matrix representation

An adjacency matrix representation

- uses an array of arrays, one array per vertex;
- the i -th array stores a boolean at its j -th position:
 - ▶ the j -th boolean in array i is false if no edge from i to j ;
 - ▶ the j -th boolean in array i is true if an edge from i to j .

An example:



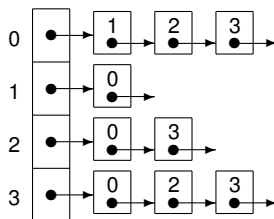
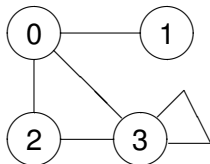
0	•	→	0	1	1	1
1	•	→	1	0	0	0
2	•	→	1	0	0	1
3	•	→	1	0	1	1

adjacency list representation

An adjacency list representation

- uses an array of lists, one list per vertex;
- the list i stores the vertices adjacent to vertex i .

An example:



The vertices in the list need not to be ordered.

The number of elements in list i equals the degree of vertex i .

comparing adjacency matrices and adjacency lists

Two drawbacks for an adjacency matrix:

- 1 For a graph with n vertices, it requires $\Theta(n^2)$ space.
- 2 Computing all edges incident to a vertex v requires $\Theta(n)$ time, as this takes the traversal of the entire row for v .

The number of edges incident to a vertex can be much smaller than n , typically it could even be $O(1)$.

Adjacency lists store all edges incident to a vertex, taking into account that the degree of a vertex could be small.

why we prefer adjacency lists

Theorem (1)

The adjacency list for a graph with n vertices and m edges requires $\Theta(n + m)$ space.

Proof. In the adjacency list representation, we have:

- 1 an array of size n , where each entry points to a list of edges; and
- 2 each list contains the edges incident to each vertex.

The array contribute $\Theta(n)$ space, while the space for the lists is $\Theta(m)$, because every edge occurs exactly twice in each list. In particular, the edge $(v, w) \in E$ appears once in the adjacency for v and another time in the adjacency list for w . Therefore, for m edges, the lists store in total $2m$ elements, and $2m$ is $\Theta(m)$.

$\Theta(n) + \Theta(m) = \Theta(n + m)$ completes the proof.

Q.E.D.

$2m$ and the sum of degrees

Proposition (2)

Let $G = (V, E)$ be a graph, $m = \#E$, then $\sum_{v \in V} \deg(v) = 2m$.

Proof. For a vertex $v \in V$, the degree $\deg(v)$ counts the number of edges incident to v .

In the adjacency edge list representation of G , every edge $(v, w) \in E$ appears exactly twice.

In $\sum_{v \in V} \deg(v)$, every edge $(v, w) \in E$ is also counted exactly twice:
once in $\deg(v)$ and once in $\deg(w)$.

As $\#E = m$, therefore: $\sum_{v \in V} \deg(v) = 2m$.

Q.E.D.

Implementing Graph Traversals

1 Data Structures for Graphs

- representing graphs
- adjacency matrices and adjacency lists

2 Implementing the Breadth-First and Depth-First Search Algorithms

- implementing the breadth-first search algorithm
- implementing the depth-first search algorithm

3 Applications

- connected components
- bipartiteness testing

layers computed by the breadth-first search algorithm

Starting at a node in the graph, the breadth-first search algorithm explores all possible directions, computing one new layer in each step.

Consider a graph $G = (V, E)$, with root node r .

- $L_0 = \{ r \}$ is layer zero.
- $L_1 = \{ s \in V \setminus \{r\} \mid (r, s) \in E \}$ is the first layer.
- $L_2 = \{ s \in V \setminus \{r\} \setminus L_1 \mid (t, s) \in E, t \in L_1 \}$ is the second layer.

We generalize L_2 in the following definition.

Definition (3)

In a graph $G = (V, E)$, where some $r \in V$ is the root node in the breadth-first search, **the k -th layer** L_k is $L_0 = \{r\}$, and for $k > 1$:

$$L_k = \left\{ s \in V \setminus \{r\} \setminus \bigcup_{i=1}^{k-1} L_i \mid (t, s) \in E, t \in L_{k-1} \right\}.$$

the breadth-first search tree of a graph

Given a vertex $r \in V$ of a graph $G = (V, E)$,
the breadth-first search returns the component of G connected to r .

The vertices in the component connect to r are organized in a tree.

Definition (4)

The *breadth-first search tree of a graph* contains all the vertices of the graph and the edges visited in the breadth-first search.

storing a tree

The data structure for a tree is an adjacency list representation:
for every vertex that is not the root nor a leaf,
the adjacency list stores two edges:

- 1 the edge to its parent (at first position); and
- 2 the edge to its child (at second position).

The layers computed by the breadth-first search store
for each vertex the siblings in the tree.

distance between two vertices in a graph

Definition (5)

In the graph $G = (V, E)$, *the distance between two vertices* $u, v \in V$ is

- either ∞ if there is no path from u to v ; or
- is the number of edges in the shortest path between u and v .

The breadth-first search tree defines all shortest paths from the root to all other nodes in the tree.

Theorem (6)

Layer L_k contains the points at distance k from the root node in the breadth-first search.

Exercise 1: write a proof for the above theorem.

the breadth-first search algorithm — initialization

Input: $G = (V, E)$ with $r \in V$ as the root node.

Output: L_k the layers of vertices, $L_0 = \{ r \}$; and

$T = (V_T, E_T)$ the breadth-first search tree, rooted at r .

We keep track of the vertices that are visited:

```
visited[r] := true
for all  $v \in V \setminus \{ r \}$  do
    visited[v] := false
```

```
 $k := 0$ 
 $L_0 := \{ r \}$ 
 $L_1 := \emptyset$ 
 $V_T := \{ r \}$ 
 $E_T := \emptyset$ 
```

the breadth-first search algorithm — the loop

```
do
  for all  $v \in L_k$  do
    for all  $(v, w) \in E$  do
      if not visited[w] then
        visited[w] := true
         $V_T := V_T \cup \{ w \}$ 
         $E_T := E_T \cup (v, w)$ 
         $L_{k+1} := L_{k+1} \cup \{ w \}$ 
  if  $L_{k+1} = \emptyset$  then
    exit the do loop
else
   $k := k + 1$ 
   $L_{k+1} := \emptyset$ 
```

data structures for the breadth-first search

On input:

- The graph G is given by its adjacency list representation.

On output:

- The breadth-first search tree T is returned as adjacency lists.
- The `visited` is an array of size n .
- The layers L are an array of lists:
 - 1 L is an array of size n ; and
 - 2 each L_k is a list of vertices.

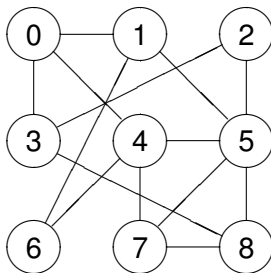
Lemma (7)

The data structures T , `visited`, and L occupy $\Theta(n)$ space.

Exercise 2: Write a proof for the Lemma.

another exercise

Consider the graph:



Exercise 3:

- 1 Write the adjacency list representation for the graph.
- 2 Run the breadth-first search algorithm and give the evolution of the data structures T , visited , and L in each step.

cost of the breadth-first search algorithm

Theorem (8)

If $G = (V, E)$, $n = \#V$ and $m = \#E$ is given by an adjacency list representation, then the breadth-first search runs in $O(n + m)$.

Proof. The initialization is $O(n)$, as we go through every element of `visited`, an array of size n . The other initializations are $O(1)$.

From the double loop, for $v \in L_k$, for all $(v, w) \in E$, we obtain $O(nm)$, as $\#E = m$ and $L_k \subset V$, $\#V = n$. But observe:

- 1 Every vertex v occurs *only once* in some layer L_k .
- 2 The length of the adjacency list for v is the degree of v .

Therefore, the number of steps by the double loop is $\sum_{v \in V} \deg(v)$.

By the $v \in V$ in the sum, the sum bounds the total number of steps in *all* executions of the double loop during the algorithm.

proof continued

By the Proposition (2), $\sum_{v \in V} \deg(v) = 2m$, which is $O(m)$.

Examine the cost of manipulating the data structures:

- Accessing an element in `visited` is $O(1)$.
- Inserting an element to the list L_{k+1} is $O(1)$.
- Inserting a new edge to the adjacency list representation of T is $O(1)$.

The initialization is $O(n)$, the loop runs in $O(m)$, and each step in the loop is $O(1)$.

Therefore, breadth-first search runs in $O(n + m)$.

Q.E.D.

queues

If we do not care about the layers L_k and remove them from the output of the breadth-first search algorithm, then we can work with two queues:

- The queue of the current layer L_k .
As we do for $v \in L_k$, v is popped from the queue L_k .
- The queue for the next layer L_{k+1} .
For all $(v, w) \in E$, with $v \in L_k$, w is appended to the queue L_{k+1} .

We can call the queues `current` and `next`.

At the end of each iteration in the loop:

- The queue `current`, L_k , is empty.
- The queue `next`, L_{k+1} , becomes the `current` queue.

Exercise 4: Give an example of a graph with n vertices where the queue `next` has size $n - 1$.

Implementing Graph Traversals

1 Data Structures for Graphs

- representing graphs
- adjacency matrices and adjacency lists

2 Implementing the Breadth-First and Depth-First Search Algorithms

- implementing the breadth-first search algorithm
- implementing the depth-first search algorithm

3 Applications

- connected components
- bipartiteness testing

CS 401/MCS 401 Lecture 4
Computer Algorithms I
Jan Verschelde, 24 June 2026

Implementing Graph Traversals

1 Data Structures for Graphs

- representing graphs
- adjacency matrices and adjacency lists

2 Implementing the Breadth-First and Depth-First Search Algorithms

- implementing the breadth-first search algorithm
- implementing the depth-first search algorithm

3 Applications

- connected components
- bipartiteness testing

finding the exit in a maze

- The Breath-First Search (BFS) algorithm layers the nodes in a graph in a visit of all connected nodes. We first visit all neighbors of the current node before going to the neighbors of the neighbors.

We compute all shortest paths from the root vertex of the BFS tree to all other nodes in the connected component of the graph.

- The Depth-First Search (DFS) algorithm is best applied to finding the exit in a maze, or escaping a burning building.

As soon as the DFS algorithm gets to a new neighbor w , it starts to explore the first neighbor of w .

Eventually, if the path that starts at w does not lead to the exit, the DFS algorithm *backtracks* to the next sibling of w , the node in the same layer as w .

stacks

The depth-first search algorithm has a natural recursive formulation.

A stack is a natural data structure for a recursive algorithm.

The stack is initialized with the root $r \in V$ of the depth-first search tree.

In the loop of the depth-first search algorithm:

- 1 We start by popping a vertex v from the stack.
- 2 All vertices adjacent to v are pushed onto the stack.

The outcome of the algorithm is a tree.

Definition (9)

The *depth-first search tree of a graph* contains all the vertices of the graph and the edges visited in the depth-first search.

The branches in a depth-first search tree tend to be longer than the branches in a breadth-first search tree.

a first outline of the depth-first search algorithm

Input: $G = (V, E)$ with $r \in V$ as the root node.

Output: $W \subseteq V$, $w \in W$: there is a path from r to w .

$W := \emptyset$

for all $v \in V$ do

$\text{explored}[v] := \text{false}$

push r to the stack S

while $S \neq \emptyset$ do

 pop v from the stack S

 if not $\text{explored}[v]$ then

 for each $(v, w) \in E$ do

 push w to the stack S

$W := W \cup \{v\}$

$\text{explored}[v] := \text{true}$

constructing the depth-first search tree

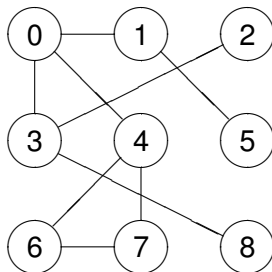
To construct the depth-first search tree we need to know which vertex v caused the vertex w to be pushed to the stack so that we return to w and set `explored[ $w$ ]` to true, we can then add the edge (v, w) to the tree.

For the backtracking, when returning, we need an additional array `parent` to record the parent for each vertex w we push to the stack.

- After pushing w to the stack, we set v to `parent[ $w$ ]`.
- When `explored[ $v$ ] := true`, then we add $(v, \text{parent}[v])$ to the tree, except of course in case v is the root node.

defining the array `parent`

Consider the graph:



Exercise 5:

Run the depth-first search algorithm on the graph above and give the evolution of `explored`, the `stack`, and `parent` in each step.

complete depth-first search algorithm — initialization

Input: $G = (V, E)$ with $r \in V$ as the root node.

Output: $W \subseteq V$, $w \in W$: there is a path from r to w .

$T = (V_T, E_T)$ the depth-first search tree, rooted at r .

$W := \emptyset$

$V_T := \{ r \}$

$E_T := \emptyset$

for all $v \in V$:

 explored[v] := false

push r to the stack S

complete depth-first search algorithm — the loop

Input: $G = (V, E)$ with $r \in V$ as the root node.

Output: $W \subseteq V$, $w \in W$: there is a path from r to w .

$T = (V_T, E_T)$ the depth-first search tree, rooted at r .

```
while  $S \neq \emptyset$  do
  pop  $v$  from the stack  $S$ 
  if not explored[ $v$ ] then
    for each  $(v, w) \in E$  do
      push  $w$  to the stack  $S$ 
      parent[ $w$ ] :=  $v$ 
     $W := W \cup \{v\}$ 
  explored[ $v$ ] := true
  if  $v \neq r$  then
     $V_T := V_T \cup \{v\}$ 
     $E_T := E_T \cup (v, \text{parent}[v])$ 
```

cost of the depth-first search algorithm

Theorem (10)

If $G = (V, E)$, $n = \#V$ and $m = \#E$ is given by an adjacency list representation, then the depth-first search runs in $O(n + m)$.

Proof. The cost of the initialization of `explored` is $O(n)$. Pushing and popping elements takes $O(1)$ time. To bound the running time we need to count the number of elements on the stack. A vertex is added to the stack each time one of its adjacent vertices is explored. Applying Proposition (2), the total number of additions is thus

$$\sum_{v \in V} \deg(v) = 2m.$$

Inserting a vertex in T and accessing `parent` is $O(1)$. The cost of the initialization is $O(n)$ and the number of stack operations is $O(m)$, which leads to $O(n + m)$.

Q.E.D.

Implementing Graph Traversals

1 Data Structures for Graphs

- representing graphs
- adjacency matrices and adjacency lists

2 Implementing the Breadth-First and Depth-First Search Algorithms

- implementing the breadth-first search algorithm
- implementing the depth-first search algorithm

3 Applications

- **connected components**
- bipartiteness testing

the set of connected components

Consider the following algorithm:

- 1 Both the breadth-first and depth-first search algorithm compute the set of all vertices connected to a vertex in $O(n + m)$ for a graph with n vertices and m edges.
If a graph is connected, then one search suffices.
- 2 If the graph is not connected, then there is some $v \in V$ which was not visited by the breadth-first search or not in the list of explored vertices of the depth-first search.
- 3 Then take v as the start vertex in another search and go to step 1.

The cost of this algorithm is $O(n(n + m))$ because we have n vertices and each run of the search algorithm is $O(n + m)$.

Can we derive a sharper upper bound on the running time?

the structure of the set of connected components

Lemma (11)

For any two vertices v and w in a graph, their connected components are either identical or disjoint.

Proof. There are two cases to consider.

- 1 There is a path between v and w .

If we apply a search starting from v , then the connected component will contain w . Vice versa, a search with root node w will yield a connected component that contains v .

The set of vertices in both searches will be identical.

- 2 There is no path between v and w .

proof continues ...

proof continued with the second case

Lemma (11)

For any two vertices v and w in a graph, their connected components are either identical or disjoint.

- 2 There is no path between v and w .

Assume a search starting at v contains w . In that case, there is a path from v to w , contradicting the assumption.

The same argument applies to a search starting at w . In this case, the connected components are disjoint.

The two cases prove the statement of the lemma.

Q.E.D.

computing the set of connected components

Theorem (12)

Computing the set of connected components in a graph of n vertices and m edges runs in time $O(n + m)$.

Proof. By the previous Lemma (11), the work spent in the search depends on the size of the connected component. If the graph splits up in k components, and the running time in each search is $O(n_i + m_i)$ for the i -th component, where

$$n_1 + n_2 + \cdots + n_k = n \text{ and } m_1 + m_2 + \cdots + m_k = m.$$

Because we run the same search algorithm, the same constants $c > 0$ and n_0 may be used in each $O(n_i + m_i)$ running time, so the sum of all $O(n_i + m_i)$ running times is $O(n + m)$.

Step 2 in the algorithm on slide #33 runs in time $O(n)$, and $O(n) + O(n + m)$ is $O(n + m)$.

Q.E.D.

connectivity and degree

Exercise 6:

Consider a graph G with an even number n of vertices.

If every vertex in G has degree at least $n/2$,
then G is connected. True or false?

- If true, then give a proof.
- If false, then give a counter example.

Implementing Graph Traversals

1 Data Structures for Graphs

- representing graphs
- adjacency matrices and adjacency lists

2 Implementing the Breadth-First and Depth-First Search Algorithms

- implementing the breadth-first search algorithm
- implementing the depth-first search algorithm

3 Applications

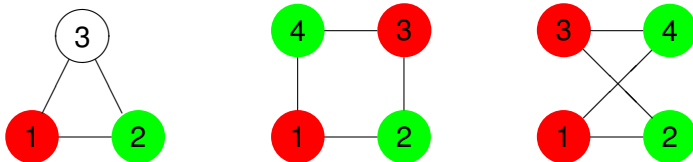
- connected components
- bipartiteness testing

bipartite graphs

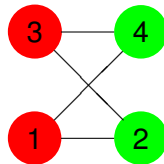
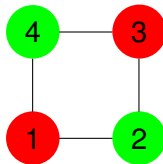
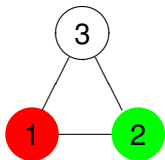
Definition (13)

A graph $G = (V, E)$ is *bipartite* if there is a partition of V in two disjoint sets U and W so that for all $u_1, u_2 \in U$: $(u_1, u_2) \notin E$, and for all $w_1, w_2 \in W$: $(w_1, w_2) \notin E$.

We can color all nodes in a bipartite graph with only two colors so all adjacent nodes are colored differently.



odd cycles



Definition (14)

An *odd cycle* in a graph is a cycle with an odd number of vertices.

Obviously, a bipartite graph has no odd cycles.

Theorem (15)

A graph is bipartite if and only if it contains no odd cycles.

The correctness of the following algorithm proves this theorem.

designing an algorithm

Assume the graph $G = (V, E)$ is connected.

- 1 Pick any node $s \in V$ and color it red.
- 2 Color all neighbors of s green.
- 3 Color all the neighbors of the neighbors of s red.

Run the breadth-first algorithm and color the vertices in layer L_k as:

- if k is even, then all vertices in L_k are red, and
- if k is odd, then all vertices in L_k are green.

If for all k , for all $u, v \in L_k$: $(u, v) \notin E$, then G is bipartite.

If G is not bipartite, then there is a k and $u, v \in L_k$: $(u, v) \in E$.

Then we have an odd cycle, of length $2k + 1$:

- 1 The path from s to u has k edges.
- 2 There is one edge from u to v .
- 3 The path from v to s has k edges.

testing if a graph is bipartite

Theorem (16)

Testing for bipartiteness for n vertices and m edges is $O(n + m)$.

Proof. The breadth-first search algorithm produces layers L_k , which define a coloring of each vertex v in the graph:

red if $v \in L_k$ and k is even, green if $v \in L_k$ and k is odd.

The color is applied when v is added to L_k , so coloring is $O(1)$.

The test whether (v, w) is an edge for a $v \in L_k$ requires the traversal of the adjacency list for v which is done only once:

for each $v \in L_k$, once L_k is computed.

This test is $\Theta(m)$, because the adjacency list requires $\Theta(m)$ space.

The breadth-first search algorithm runs in $O(n + m)$
and $O(n + m) + \Theta(m)$ is $O(n + m)$.

Q.E.D.

Is there a cycle in a graph?

Exercise 7:

Consider as given a graph G with n vertices and m edges.

- 1 Write an algorithm to decide if G contains a cycle.
- 2 Show that your algorithm runs in time $O(n + m)$.